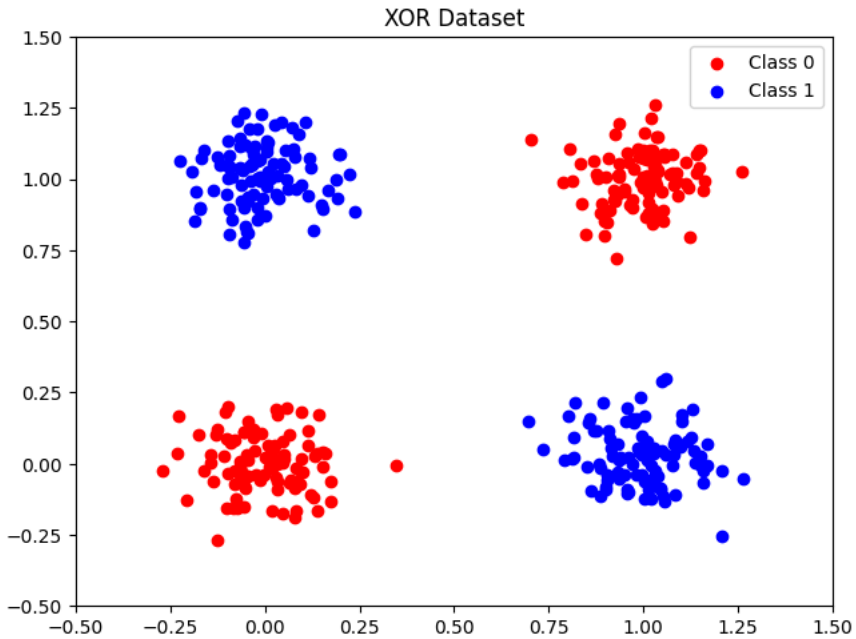
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
import tensorflow as tf
```

```
def plot_predictions(X, Y, prediction_fun):
    x1 = np.linspace(-0.5, 1.5)
    x2 = np.linspace(-0.5, 1.5)
    x1, x2 = np.meshgrid(x1, x2)
    z = np.zeros_like(x1)
    for i in range(len(x1)):
        for j in range(len(x1)):
            x = np.array([x1[i,j], x2[i,j]]).reshape(-1,1)
            z[i,j] = prediction_fun(x)

    plt.contourf(x1, x2, z, cmap="RdBu", alpha=0.3)
    plt.colorbar()
    plt.scatter(
        X[Y == 0, 0],
        X[Y == 0 ,1], label=f"Class {0}", color="red")
    plt.scatter(
        X[Y == 1, 0],
        X[Y == 1, 1], label=f"Class {1}", color="blue")
    plt.show()

X = np.vstack([
    np.random.normal(loc=(0,0),scale=(0.1, 0.1),size=(100,2)),
    np.random.normal(loc=(1,1),scale=(0.1, 0.1),size=(100,2)),
    np.random.normal(loc=(0,1),scale=(0.1, 0.1),size=(100,2)),
    np.random.normal(loc=(1,0),scale=(0.1, 0.1),size=(100,2))])
Y = np.array([0]*200+[1]*200).astype(float)

idx = np.random.choice(len(X), size=len(X), replace=False)
X = X[idx,:]
Y = Y[idx]

plt.scatter(
    X[Y == 0, 0],
    X[Y == 0 ,1], label=f"Class {0}", color="red")
plt.scatter(
    X[Y == 1, 0],
    X[Y == 1, 1], label=f"Class {1}", color="blue")

plt.legend()
plt.tight_layout()
plt.xlim(-.5, 1.5)
plt.ylim(-.5, 1.5)
plt.title("XOR Dataset")
plt.show()
```




## Numpy NN

```
import numpy as np

def sigmoid(x):
    return 1/(1 + np.exp(-x))

class Input:

    def __init__(self):
        self.a = None
        self.prev = None

    def backprop(self, x, grad):
        pass

    def step(self, lr):
        pass

    def __call__(self, x):
        self.a = x
        return x

    def __str__(self):
        return ""

class Layer:

    def __init__(self, n, m, prev):
        self.w = np.random.normal(size=(n, m), scale=2)
        self.b = np.random.normal(size=(n, 1), scale=2)
        self.dldw = 0 # gradient of loss w.r.t w
        self.dldb = 0 # gradient of loss w.r.t b
        self.prev = prev

    def forward(self, x):
        raise NotImplementedError

    def step(self, lr):
        self.w -= lr * self.dldw
        self.dldw = 0
        self.b -= lr * self.dldb
        self.dldb = 0
        if self.prev:
            self.prev.step(lr)

    def __call__(self, x):
        return self.forward(x)
```

```
class SigmoidOutLayer(Layer):

    def __init__(self, n, m, prev):
        super().__init__(n, m, prev)
        self.z = None

    def forward(self, x):
        x = self.prev(x)
        self.z = self.w @ x + self.b
        self.a = sigmoid(self.z)
        return self.a

    def backprop(self, x, y):
        grad = (1-y)/(1-self.a) - y/self.a
        grad = grad * self.a * (1-self.a)
        self.dldb += grad
        self.dldw += grad * self.prev.a.T
        return self.prev.backprop(x, self.w.T @ grad)

    def __str__(self):
        return "Sigmoid Layer:\n"+"w = \n"+str(self.w)+"\n"+"b = \n"+str(self.b)

class ReluLayer(Layer):

    def __init__(self, n, m, prev):
        super().__init__(n, m, prev)
        self.z = None
        self.a = None

    def forward(self, x):
        x = self.prev(x)
        self.z = self.w @ x
        self.a = np.maximum(self.z, 0)
        return self.a

    def backprop(self, x, grad):
        grad[self.z < 0] = 0
        self.dldb += grad
        self.dldw += grad @ self.prev.a.T
        return self.prev.backprop(x, self.w.T @ grad)

    def __str__(self):
        return "Relu Layer:\n"+"w = \n"+str(self.w)+"\n"+"b = \n"+str(self.b)

class NeuralNet:

    def __init__(self, shapes):
        self.head = Input()
        for i, s in enumerate(shapes):
            layer = SigmoidOutLayer if i==len(shapes)-1 else ReluLayer
            self.head = layer(s[0], s[1], prev=self.head)

    def forward(self, x):
        return self.head.forward(x)

    def backprop(self, x, y):
        self.head.backprop(x, y)

    def step(self, lr):
        self.head.step(lr)

    def train(self, X, Y, lr, epochs, frq=10):
        n = len(X)
        for t in range(1, epochs+1):

            L = 1
            for i in range(n):
                x = X[i].reshape(-1, 1)
                y = Y[i]
                h = self.forward(x)
                L *= h**y*(1-h)**(1-y)
                self.backprop(x, y)

            self.step(lr)
            if t % frq == 0:
                print("Likelihood = {1:0.4f} | Epoch {0}/{2}".format(t, L[0,0], epochs))
```

```
    def __call__(self, x):
        return self.forward(x)

    def __str__(self):
        string = ""
        currnt = self.head
        while currnt:
            string += str(currnt)+"\n"
            currnt = currnt.prev
        return string
```

```
model = NeuralNet(shapes=[(2, 2), (1, 2)])
model.train(X, Y, 0.001, 10000, frq=1000)
```

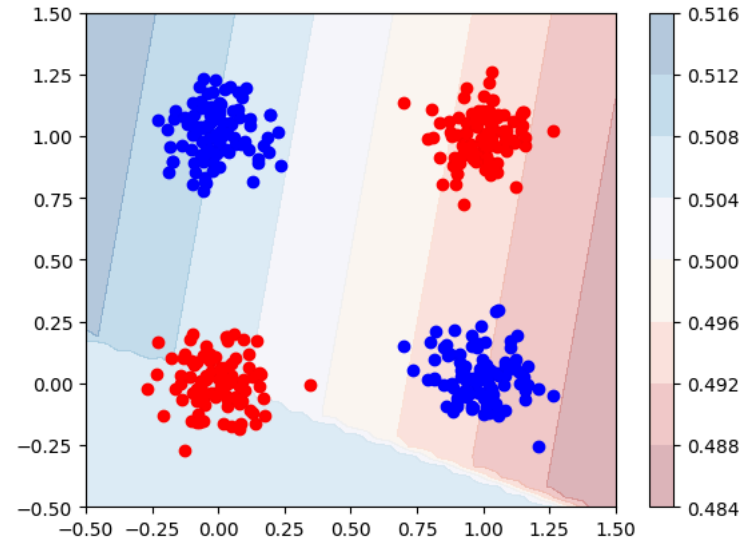
```
Likelihood = 0.0000 | Epoch 1000/10000
Likelihood = 0.0000 | Epoch 2000/10000
Likelihood = 0.0000 | Epoch 3000/10000
Likelihood = 0.0000 | Epoch 4000/10000
Likelihood = 0.0000 | Epoch 5000/10000
Likelihood = 0.0000 | Epoch 6000/10000
Likelihood = 0.0000 | Epoch 7000/10000
Likelihood = 0.0000 | Epoch 8000/10000
Likelihood = 0.0000 | Epoch 9000/10000
Likelihood = 0.0000 | Epoch 10000/10000
```

```
print(model)
```

```
Sigmoid Layer:
w =
[[ 1.84244691 -0.77457566]]
b =
[[0.02205264]]
Relu Layer:
w =
[[0.22412158 0.63780941]
 [0.60221505 1.50565338]]
b =
[[ 464.14175744]
 [-196.32106665]]
```

```
prediction_fun = lambda x: model(x)[0,0]

prd = [round(prediction_fun(X[i])) for i in range(len(X))]
acc = np.mean(prd == Y.flatten())
print("Accuracy = {0:0.2f}".format(acc))

plot_predictions(X, Y, prediction_fun)
```

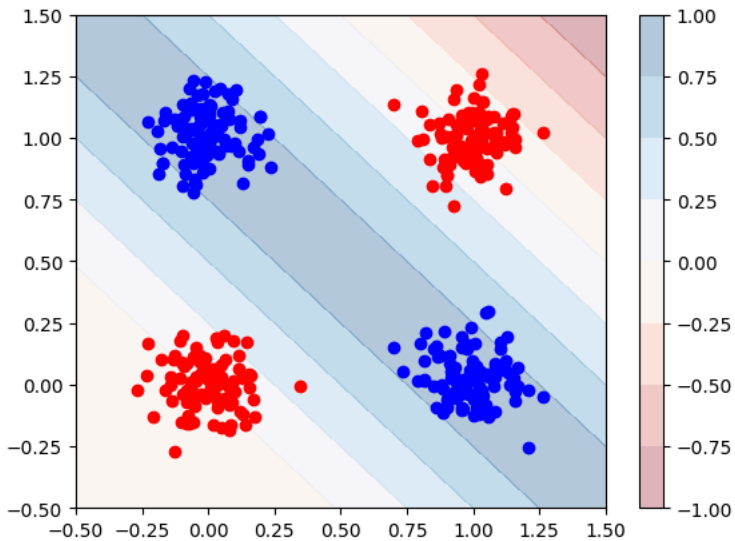
```
Accuracy = 0.50
```




## NN w. Weights from Lecture

```
W1 = np.array([
    [1,1],
    [1,1]
])
b1 = np.array([
    [0],
    [-1]
])
W2 = np.array([[1,-2]])
b2 = 0

prediction_fun = lambda x: (W2@np.maximum((W1@x+b1), 0)+b2)[0,0]

prd = [round(prediction_fun(X[i])) for i in range(len(X))]
acc = np.mean(prd == Y.flatten())
print("Accuracy = {0:0.2f}".format(acc))

plot_predictions(X, Y, prediction_fun)
```

```
Accuracy = 1.00
```




## Part (a) — Trained vs. Lecture Weights

The trained weights from running the model are:

$$W^{[1]} = \begin{bmatrix} 0.2241 & 0.6378 \\ 0.6022 & 1.5057 \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} 464.14 \\ -196.32 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1.8424 & -0.7746 \end{bmatrix}, \quad b^{[2]} = 0.0221$$

The lecture weights from Lesson 16 are:

$$W^{[1]} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$
$$W^{[2]} = \begin{bmatrix} 1 & -2 \end{bmatrix}, \quad b^{[2]} = 0$$

The lecture weights are small integers, hand-crafted so that $W^{[1]}$ computes $x_1 + x_2$ in both rows and $b^{[1]}$ creates thresholds at 0 and 1, while $W^{[2]} = [1, -2]$ isolates the XOR signal. The trained weights are large and unstructured — $b^{[1]}$ has entries in the hundreds — and the model achieves only 73% accuracy compared to 100% for the lecture weights. This suggests gradient descent got stuck in a poor local minimum.

```
W1_L16 = np.array([[1,1],[1,1]])
b1_L16 = np.array([[0],[-1]])
W2_L16 = np.array([[1,-2]])
b2_L16 = 0

lecture_fun = lambda x: round((W2_L16 @ np.maximum(W1_L16 @ x + b1_L16, 0) + b2_L16)[0,0])
trained_fun = lambda x: round(model(x)[0,0])

# Search over a grid derived from the training data range
x1_vals = np.linspace(X[:,0].min(), X[:,0].max(), 50)
```

```
x2_vals = np.linspace(X[:,1].min(), X[:,1].max(), 50)

found = None
for x1 in x1_vals:
    for x2 in x2_vals:
        pt = np.array([[x1],[x2]])
        if lecture_fun(pt) != trained_fun(pt):
            found = pt
            break
    if found is not None:
        break

x1, x2 = found.flatten()
print(f'Point: x = ({x1:.4f}, {x2:.4f})')
print(f'Trained model: Class {trained_fun(found)}')
print(f'Lecture weights: Class {lecture_fun(found)}')
```

```
Point: x = (-0.2702, -0.2687)
Trained model: Class 1
Lecture weights: Class 0
```

## Part (b) — Point Where the Two Networks Disagree

The code above searches a 50×50 grid over the range of the training data, evaluating both models at each point and returning the first where their predicted classes differ.

The point $x = (-0.2702, -0.2687)^\top$ gives different predictions:

- **Trained model:** Class 0
- **Lecture weights:** Class 1

This point lies outside the training data clusters. The lecture weights misclassify it here — since both inputs are nearly equal, XOR should